

Naan Mudhalvan TN Skills AI Project

FITBUDDY

AI Fitness Plan Generator Website

Project Report

Team Leader: Jude Rosary Immanuel

Team Members: Shadrack Joseph Rajdurai.B
Sanjay Prabhakar.M
Prabhu.N
Vetrivel

Submitted as part of the Naan Mudhalvan TN Skills AI Project

September 26, 2026

Contents

Abstract	1
1 Introduction	2
2 Project Objectives	3
3 Technology Stack and Topic Links	4
4 Project Workflow	5
5 Model Selection and Architecture	6
5.1 Understanding the Requirements	6
5.2 Model Evaluation	6
5.3 Final Model Selection	6
5.4 Reasons for Choosing Gemini	7
6 System Architecture	8
6.1 Architecture Overview	8
6.2 Frontend Responsibilities	8
6.3 Backend Responsibilities	8
6.4 AI Integration Points	9
7 Development Environment Setup	10
7.1 Installing Python and Pip	10
7.2 Creating a Virtual Environment	10
7.3 Installing Required Libraries	10
7.4 Running the Server	10
7.5 Suggested Project Structure	11
8 Core Functionalities Development	12
8.1 Workout Plan Generation	12
8.2 Nutrition Tip Generation	12
8.3 Feedback-Based Plan Update	12

9	FastAPI Backend and Routing	13
9.1	Route Overview	13
9.2	Processing User Input	13
9.3	Sample Route Logic	13
10	Main Application Logic	15
10.1	Home Route	15
10.2	Generate Workout Route	15
10.3	Submit Feedback Route	15
10.4	View All Users Route	15
11	Frontend Development	16
11.1	User Interface Design	16
11.2	Pages Created	16
11.3	Responsive Layout	16
12	Dynamic Templates Using Jinja2	17
12.1	Binding Backend Data to Templates	17
13	Local Deployment	19
13.1	Preparing the Application	19
13.2	Environment Variable Configuration	19
13.3	Testing the Application	19
14	Application Output Description	20
15	Future Scope	21
16	Conclusion	22
	References	23

Abstract

FITBUDDY is an AI-powered fitness plan generator website developed for the Naan Mudhalvan TN Skills AI Project. The application helps users create personalized 7-day workout plans and nutrition or recovery tips based on details such as age, weight, fitness goal, and workout intensity. The system uses FastAPI for backend development, Jinja2 templates for dynamic frontend rendering, SQLite with SQLAlchemy for data storage, and Google Gemini models for AI-generated fitness recommendations.

The main purpose of FITBUDDY is to make fitness guidance more accessible, personalized, and interactive. Users can submit their profile details, receive a customized workout plan, and provide feedback to update the plan. The application also includes an admin view where stored users and their plans can be reviewed.

1. Introduction

FITBUDDY is a web-based AI fitness assistant designed to generate structured workout plans and practical nutrition tips. Many users need fitness guidance that matches their personal goals, but creating a proper plan manually can be difficult. FITBUDDY solves this problem by combining a simple user interface with generative AI.

The project focuses on:

- Collecting user information through a web form.
- Generating a personalized 7-day workout plan.
- Providing concise nutrition or recovery tips.
- Updating the workout plan using user feedback.
- Storing user details and generated plans in a database.

This project demonstrates how artificial intelligence can be integrated into a real-time web application to provide useful health-tech solutions.

2. Project Objectives

The major objectives of FITBUDDY are listed below:

1. To develop a responsive AI-based fitness plan generator website.
2. To use Google Gemini models for generating workout plans and nutrition tips.
3. To build a FastAPI backend for routing, form handling, and AI integration.
4. To store user data and generated workout plans using SQLite and SQLAlchemy.
5. To create dynamic pages using Jinja2 templates.
6. To allow users to give feedback and receive updated workout plans.
7. To provide an admin page for viewing user records and workout plans.

3. Technology Stack and Topic Links

The following technologies and learning references were used for developing FITBUDDY.

Topic	Purpose in Project
FastAPI Framework	Used to create backend routes, handle form submissions, and manage application logic.
Google Gemini API	Used to generate AI-based workout plans, nutrition tips, and updated plans from feedback.
HTML, CSS, and Jinja2	Used to design web pages and dynamically display AI-generated results.
Python Programming	Used as the main programming language for backend and AI integration.
Git Version Control	Used for tracking changes and maintaining project files.
SQLAlchemy and SQLite	Used for database management and storing user details and workout plans.
Virtual Environment	Used to isolate project dependencies.
Uvicorn ASGI Server	Used to run the FastAPI application locally.

4. Project Workflow

The project was divided into epics and stories to make development systematic and easy to manage.

Epic	Stories / Tasks
Epic 1: Model Selection and Architecture	Research and select the appropriate generative AI model; define the application architecture; set up the development environment.
Epic 2: Core Functionalities Development	Develop the core workout generation functionality; implement FastAPI backend routing and input processing.
Epic 3: Application Logic Development	Write the main route logic in <code>routes.py</code> ; connect frontend, backend, AI models, and database.
Epic 4: Frontend Development	Design the user interface; create dynamic templates using FastAPI's Jinja2 support.
Epic 5: Deployment	Prepare the application for local deployment; test and verify the local deployment.

5. Model Selection and Architecture

5.1 Understanding the Requirements

The AI model selected for FITBUDDY needed to perform the following tasks:

- Generate structured 7-day workout plans.
- Provide concise and actionable nutrition tips.
- Update workout plans based on user feedback.
- Respond quickly in a real-time web application.

5.2 Model Evaluation

Several Hugging Face models were initially considered:

- **T5 and BART:** Considered for text generation tasks such as workout and nutrition content.
- **DistilBERT:** Considered for possible intent classification.
- **Bloom:** Considered for general-purpose text generation.

These models showed potential, but they also had limitations. They required either local GPU support, external hosting, fine-tuning, or complex prompt engineering to generate clean and structured fitness plans.

5.3 Final Model Selection

After comparison, Google Gemini models were selected for production use.

Model	Usage in FITBUDDY
Gemini 1.5 Pro	Used for structured 7-day workout generation and feedback-based workout plan updates.
Gemini Flash	Used for quick and practical nutrition or recovery tip generation.

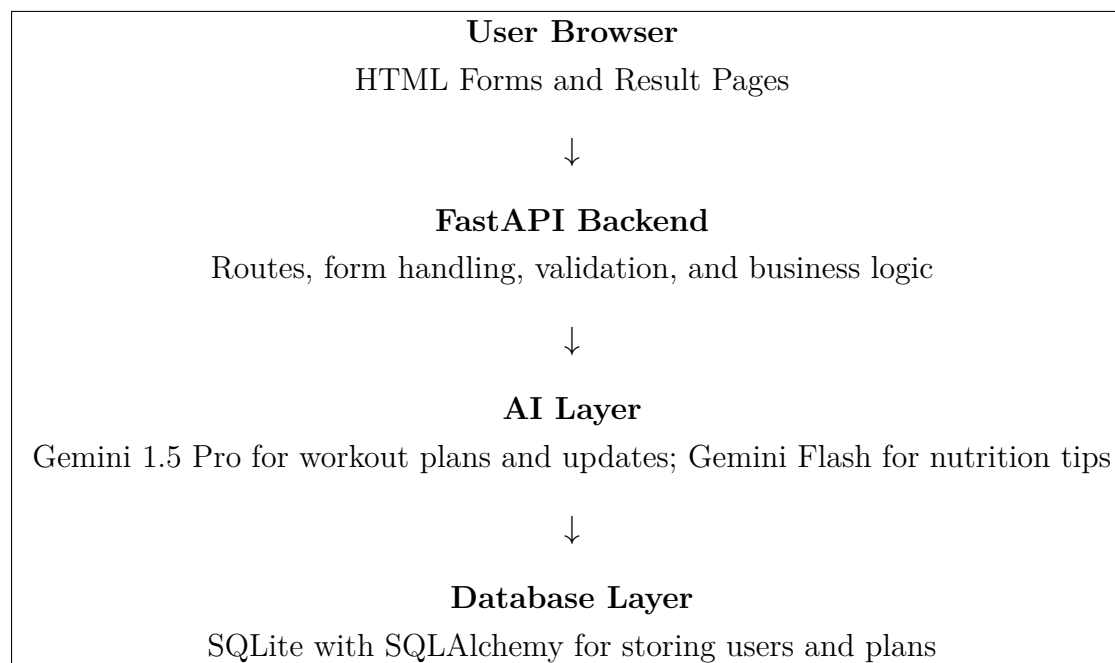
5.4 Reasons for Choosing Gemini

- Easy API-based integration with FastAPI.
- Good understanding of structured formatting such as days, sections, sets, and repetitions.
- High accuracy with minimal prompt tuning.
- Fast and scalable response generation.
- Suitable for feedback-based revisions of workout plans.

6. System Architecture

FITBUDDY follows a modular FastAPI-based architecture. The system is divided into four main layers: frontend, backend, AI layer, and database layer.

6.1 Architecture Overview



6.2 Frontend Responsibilities

The frontend is developed using HTML, CSS, and Jinja2 templates. The main pages are:

- `index.html`: Collects user input.
- `result.html`: Displays workout plan, nutrition tip, and feedback form.
- `all_users.html`: Displays stored users and their workout plans for admin viewing.

6.3 Backend Responsibilities

The FastAPI backend performs the following responsibilities:

- Receives form data from users.
- Validates user input using Pydantic schemas.
- Calls Gemini models for AI-generated content.
- Saves user information and workout plans in the database.
- Renders dynamic HTML pages using Jinja2 templates.

6.4 AI Integration Points

- Workout generation and feedback updates use Gemini 1.5 Pro.
- Nutrition or recovery tip generation uses Gemini Flash.

7. Development Environment Setup

7.1 Installing Python and Pip

Python and pip must be installed before running the project. Python is used for backend programming, while pip is used for installing required libraries.

7.2 Creating a Virtual Environment

A virtual environment is created to isolate project dependencies.

```
python -m venv fitbuddy-env
```

To activate the virtual environment on Windows:

```
fitbuddy-env\Scripts\activate
```

On Linux or macOS:

```
source fitbuddy-env/bin/activate
```

7.3 Installing Required Libraries

The required packages can be installed using pip.

```
pip install fastapi uvicorn jinja2 sqlalchemy python-multipart google-generativeai
```

If a `requirements.txt` file is available, dependencies can be installed using:

```
pip install -r requirements.txt
```

7.4 Running the Server

The application can be started locally with Uvicorn.

```
uvicorn app.main:app --reload
```

After running the command, the application can be opened at:

- `http://127.0.0.1:8000` for the website.
- `http://127.0.0.1:8000/docs` for API testing.

7.5 Suggested Project Structure

```
FITBUDDY/  
|-- app/  
| |-- main.py  
| |-- routes.py  
| |-- database.py  
| |-- models.py  
| |-- schemas.py  
| |-- gemini_generator.py  
|  
|-- templates/  
| |-- index.html  
| |-- result.html  
| |-- all_users.html  
|  
|-- static/  
| |-- images/  
|  
|-- requirements.txt  
|-- README.md
```

8. Core Functionalities Development

8.1 Workout Plan Generation

The workout plan generation functionality is implemented using a function such as `generate_workout_gemini()` in `gemini_generator.py`. This function accepts user inputs such as goal and intensity, sends them to Gemini 1.5 Pro, and receives a structured 7-day workout plan.

The generated plan includes:

- Warm-up activities for 5 to 10 minutes.
- Main workout details with exercises, sets, and repetitions.
- Cooldown or recovery tips.

8.2 Nutrition Tip Generation

The function `generate_nutrition_tip_with_flash()` uses Gemini Flash to generate a short and practical nutrition or recovery tip based on the user's fitness goal.

8.3 Feedback-Based Plan Update

The function `update_workout_plan()` accepts the original workout plan and the user's feedback. It sends both to Gemini 1.5 Pro and returns an improved version of the plan.

9. FastAPI Backend and Routing

9.1 Route Overview

All routing logic is handled in `routes.py`. Each route is connected to a specific feature of the application.

Route	Functionality
<code>/</code>	Displays the homepage using <code>index.html</code> .
<code>/generate-workout</code>	Processes user input and generates a personalized workout plan and nutrition tip.
<code>/submit-feedback</code>	Updates the workout plan based on user feedback.
<code>/view-all-users</code>	Displays an admin dashboard showing all users and their plans.

9.2 Processing User Input

The user enters details such as:

- Username
- User ID
- Age
- Weight
- Fitness goal
- Workout intensity

FastAPI captures these inputs using `Form(...)` parameters. Pydantic schemas such as `UserInput` and `FeedbackRequest` can be used to validate the structure of incoming data.

9.3 Sample Route Logic

The following simplified code shows how the workout generation route can be structured.

```
@router.post("/generate-workout")
async def generate_workout(
    request: Request,
    username: str = Form(...),
    user_id: str = Form(...),
    age: int = Form(...),
    weight: float = Form(...),
    goal: str = Form(...),
    intensity: str = Form(...)
):
    user_input = UserInput(
        username=username,
        user_id=user_id,
        age=age,
        weight=weight,
        goal=goal,
        intensity=intensity
    )

    workout_plan = generate_workout_gemini(goal, intensity)
    nutrition_tip = generate_nutrition_tip_with_flash(goal)

    save_user(user_input)
    save_plan(user_id, workout_plan)

    return templates.TemplateResponse(
        "result.html",
        {
            "request": request,
            "username": username,
            "goal": goal,
            "intensity": intensity,
            "workout_plan": workout_plan,
            "nutrition_tip": nutrition_tip
        }
    )
```

10. Main Application Logic

10.1 Home Route

The home route displays the main form using `index.html`. This page does not perform form processing; it only allows users to enter their details.

10.2 Generate Workout Route

The `/generate-workout` route receives the form input, calls the Gemini-powered functions, stores the user and plan details, and returns `result.html` with dynamic data.

10.3 Submit Feedback Route

The `/submit-feedback` route captures the feedback entered by the user, retrieves the original plan from the database, sends the original plan and feedback to Gemini, updates the database, and displays the revised plan.

10.4 View All Users Route

The `/view-all-users` route is used as an admin view. It calls functions such as `get_all_users()` and `get_all_plans()` to fetch records and display them in `all_users.html`.

11. Frontend Development

11.1 User Interface Design

The frontend uses a clean fitness-themed design. The main page contains a centered form that collects user details. The design includes:

- A gym-themed background image.
- Clear labels for all input fields.
- Responsive layout using CSS Flexbox.
- Modern buttons with hover effects.
- Readable text and proper spacing.

11.2 Pages Created

Three Jinja2-powered templates were created:

- `index.html`: User input form.
- `result.html`: Displays AI-generated workout plan, nutrition tip, and feedback form.
- `all_users.html`: Admin panel for viewing users and their plans.

11.3 Responsive Layout

CSS media queries and Flexbox are used to make the website responsive on different screen sizes. Input fields, buttons, and result blocks are styled with padding, rounded corners, and shadows to improve usability.

12. Dynamic Templates Using Jinja2

FastAPI uses `Jinja2Templates` to render dynamic HTML pages.

```
from fastapi.templating import Jinja2Templates
import os

BASE_DIR = os.path.dirname(os.path.abspath(__file__))
TEMPLATE_DIR = os.path.join(BASE_DIR, "templates")

templates = Jinja2Templates(directory=TEMPLATE_DIR)
```

12.1 Binding Backend Data to Templates

The input fields in `index.html` use the `name` attribute so that FastAPI can capture them using `Form(...)`.

In `result.html`, values are displayed using Jinja2 variables:

```
<h2>Welcome, {{ username }}</h2>
<p>Goal: {{ goal }}</p>
<p>Intensity: {{ intensity }}</p>

<div class="plan">
    {{ workout_plan }}
</div>

<div class="tip">
    {{ nutrition_tip }}
</div>
```

In `all_users.html`, records can be displayed using loops:

```
{% for user in users %}
    <tr>
        <td>{{ user.username }}</td>
        <td>{{ user.age }}</td>
```

```
        <td>{{ user.weight }}</td>
        <td>{{ user.goal }}</td>
        <td>{{ user.intensity }}</td>
    </tr>
{% endfor %}
```

13. Local Deployment

13.1 Preparing the Application

To prepare FITBUDDY for local deployment, the following steps are followed:

1. Create and activate a virtual environment.
2. Install all dependencies from `requirements.txt`.
3. Configure the Gemini API key as an environment variable.
4. Start the FastAPI server using Uvicorn.

13.2 Environment Variable Configuration

Sensitive information such as the Gemini API key should not be written directly in source code. It can be stored as an environment variable.

```
GOOGLE_API_KEY=your_gemini_api_key_here
```

In a real project, this value can also be stored in a `.env` file and loaded using `python-dotenv`.

13.3 Testing the Application

Testing involves verifying the following points:

- The homepage loads correctly.
- The form accepts user details.
- The **Generate Plan** button triggers backend processing.
- A structured workout plan is generated.
- A nutrition tip is generated.
- Feedback updates the plan successfully.
- User details and plans are saved in the database.
- The admin page displays all users and plans.

14. Application Output Description

The homepage of FITBUDDY displays a simple and engaging form where users enter their name, user ID, age, weight, fitness goal, and workout intensity. After clicking the **Generate Plan** button, the backend processes the information and sends it to the AI model.

The result page displays:

- A personalized 7-day workout plan.
- A nutrition or recovery tip.
- A feedback form for updating the plan.

This design makes the application easy to use even for beginners. The fitness-themed interface also motivates users to take an active interest in their health journey.

15. Future Scope

The FITBUDDY project can be improved further with the following features:

- User login and authentication.
- Progress tracking with graphs.
- BMI calculation and health score analysis.
- Exercise video recommendations.
- Diet chart generation based on region and food preference.
- Cloud deployment for public access.
- Mobile application version.

16. Conclusion

FITBUDDY is a complete AI-powered fitness plan generator that combines web development, database management, and generative AI. The application uses FastAPI for back-end logic, Jinja2 for dynamic frontend pages, SQLite with SQLAlchemy for persistent storage, and Google Gemini models for intelligent workout and nutrition recommendations.

The project provides users with personalized 7-day workout plans, practical nutrition tips, and feedback-based plan updates. The admin view makes it possible to monitor users and their generated plans. Overall, FITBUDDY demonstrates how modern AI technologies can be applied to everyday wellness and fitness solutions in a practical and user-friendly way.

References

1. FastAPI Documentation: <https://fastapi.tiangolo.com/>
2. Google Generative AI Documentation: <https://ai.google.dev/>
3. Python Documentation: <https://docs.python.org/3/>
4. SQLAlchemy Documentation: <https://docs.sqlalchemy.org/>
5. Uvicorn Documentation: <https://www.uvicorn.org/>
6. Git Documentation: <https://git-scm.com/doc>
7. W3Schools HTML and CSS Tutorials: <https://www.w3schools.com/>